



Quick Sort

Akanksha

Assistant Professor

Dept. of CEA, GLA University,
Mathura

Quicksort / Partition-Exchange Sort

- Quick sort is a divide and conquer algorithm.
- It was developed by British Computer Scientist Tony Hoare and published in 1961.
- When implemented well, it can be somewhat faster than merge sort and above two or three times faster than heapsort.
- Quick sort first divides a large list into two smaller sub-lists: the low elements and the high elements. Quick sort can then recursively sort the sub-lists.

How Quick Sort Works

The steps are:

1. If the array has one or no elements, it is already sorted. Return the array. (Base Case)
2. Choose a pivot element from the array. Common strategies include selecting the first, last, middle, or a random element.
3. Partition the array into two subarrays:
 1. Left subarray: Elements less than the pivot.
 2. Right subarray: Elements greater than or equal to the pivot.
4. Recursively apply Quick Sort to the left and right subarrays. Go to Step 1 for each subarray.
5. Combine the sorted subarrays and the pivot into one sorted array.



Algorithm for Quick Sort

Quick_sort(arr, first, last)

Step 1 : $i = \text{first}$

$j = \text{last}$

$k = \text{first}$

Step 2: Repeat step 3 to 5 while $(i < j)$

Step 3: (i) Set $i = i + 1$

(ii) while $\text{arr}[i] < \text{arr}[k]$

Set $i = i + 1$

Step 4: while $(\text{arr}[j] > \text{arr}[k])$

Set $j = j - 1$

Step 5: if $(i < j)$

(i) Set $\text{temp} = \text{arr}[i]$

(ii) Set $\text{arr}[i] = \text{arr}[j]$

(iii) Set $\text{arr}[j] = \text{temp}$

Step 6: (i) Set $\text{temp} = \text{arr}[k]$

(ii) Set $\text{arr}[k] = \text{arr}[j]$

(iii) Set $\text{arr}[j] = \text{temp}$

Step 7: if $(\text{first} < j - 1)$

Quick_sort(arr, first, $j - 1$)

Step 8: if $(j + 1 < \text{last})$

Quick_sort(arr, $j + 1$, last)

Step 9: Stop

Choose 31 as the pivot



①

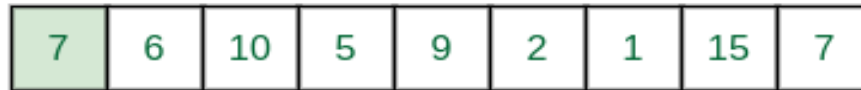
31	98	13	57	91	20	2	63
----	----	----	----	----	----	---	----

Recursively sort subsequence
on each side of pivot



②

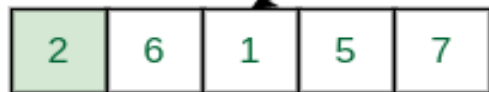
13	20	2	31	63	91	57	98
----	----	---	----	----	----	----	----



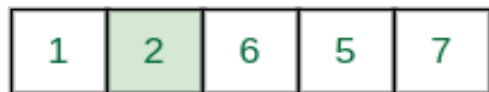
Partition around the first element 7



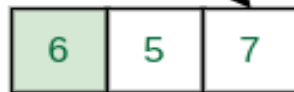
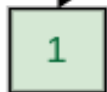
Correct position of 7



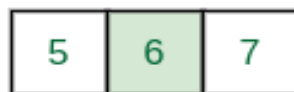
Partition around the first element 2



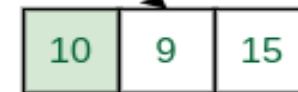
Correct position of 2



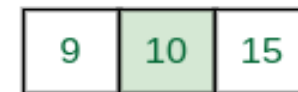
Partition around the first element 6



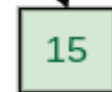
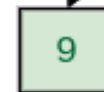
Correct position of 6



Partition around the first element 10



Correct position of 10



X[] 8 4 3 1 6 7 11 9 2 10 5 pivot = 5

4 3 1 2 5 8 6 7 11 9 10

1 2 4 3 8 6 7 9 10 11

{ } 3 4 8 6 7 9 { }

6 7 8

X[] 1 2 3 4 5 6 7 8 9 10 11

Quick Sort Analysis

- Depends on structure or content?
 - Both
- Internal/External sort algorithm?
 - Internal
- Stable/Unstable sort algorithm?
 - Unstable
- Best Case Complexity
 - $O(n \log n)$
- Worst Case Complexity?
 - $O(n^2)$
- Algorithm Approach?
 - Divide and Conquer

Complexity Analysis of Quick Sort

- **Time Complexity:**
- **Best Case : $\Omega (N \log (N))$:**

The array is always divided into two equal halves by the pivot.
- **Average Case: $\theta (N \log (N))$**

On average, the array is evenly divided by the pivot, resulting in efficient sorting.

Complexity Analysis of Quick Sort

- **.Worst Case:** $O(N^2)$

This occurs when the pivot is the smallest or largest element, resulting in unbalanced partitions.

- **Auxiliary Space:**

$O(1)$, if we don't consider the recursive stack space.

If we consider the recursive stack space then, in the worst case quicksort could make $O(N)$.

Practice Questions

1. What is the complexity of Quicksort Algorithm? Apply the Quick Sort on the following Array elements:

23 45 12 4 67 50 26

2. What is the complexity of Selection Sort Algorithm? Apply the Selection Sort on the following Array elements:

12 45 122 47 69 55 46

3. What is the complexity of Insertion Sort Algorithm? Apply the Insertion Sort on the following Array elements:

12 45 162 47 59 65 56



GLA
UNIVERSITY
MATHURA
Recognised by UGC Under Section 2(f)

Accredited with



Grade by **NAAC**

Thank You
Keep Learning...